1 Microcontrollers and the Cortex-M3 Core

A microcontroller (MCU) integrates on one chip what a desktop computer spreads across a motherboard: a CPU core, volatile working memory (SRAM, typically tens or hundreds of KB), non-volatile program storage (Flash, from which code executes directly), and on-chip peripherals such as timers, UARTs, and GPIO. One chip, one clock, and one address space replace the external memory bus, the BIOS, and the disk of a general-purpose machine.

Aspect	Desktop CPU	Microcontroller
Integration	Core only; RAM, storage, I/O on separate chips	Core, memory, and peripherals on one chip
Address space	Per-process virtual address space via an MMU	One flat physical map for code, data, and registers
Memory protection	MMU with page tables	No MMU; at most an optional MPU
Device access	Through a driver layer in the kernel	Device registers are addresses written directly

1.1 Architecture, Core, and Chip

Three layers must be kept apart when reading documentation. The **architecture** (ARMv7-M for the Cortex-M3) defines the programmer's model, the instruction set, the exception model, and the purpose of each region of the address space. The **core** (Cortex-M3) is ARM's implementation of that architecture, including a fixed set of *core peripherals*: the NVIC, the SysTick timer, the System Control Block, and an optional MPU. The **chip** or **board** is built by a silicon vendor around the licensed core and decides how much Flash and SRAM exist and which device peripherals (UARTs, timers) are connected.

Layer	Defined by	Examples of what it fixes
Architecture (ARMv7-M)	ARM	Registers, Thumb-2 instructions, exception numbers, address regions
Core (Cortex-M3)	ARM	NVIC, SysTick, SCB, MPU, 3-stage pipeline
Chip / board	Vendor	Memory sizes, device peripherals, IRQ assignment, clock

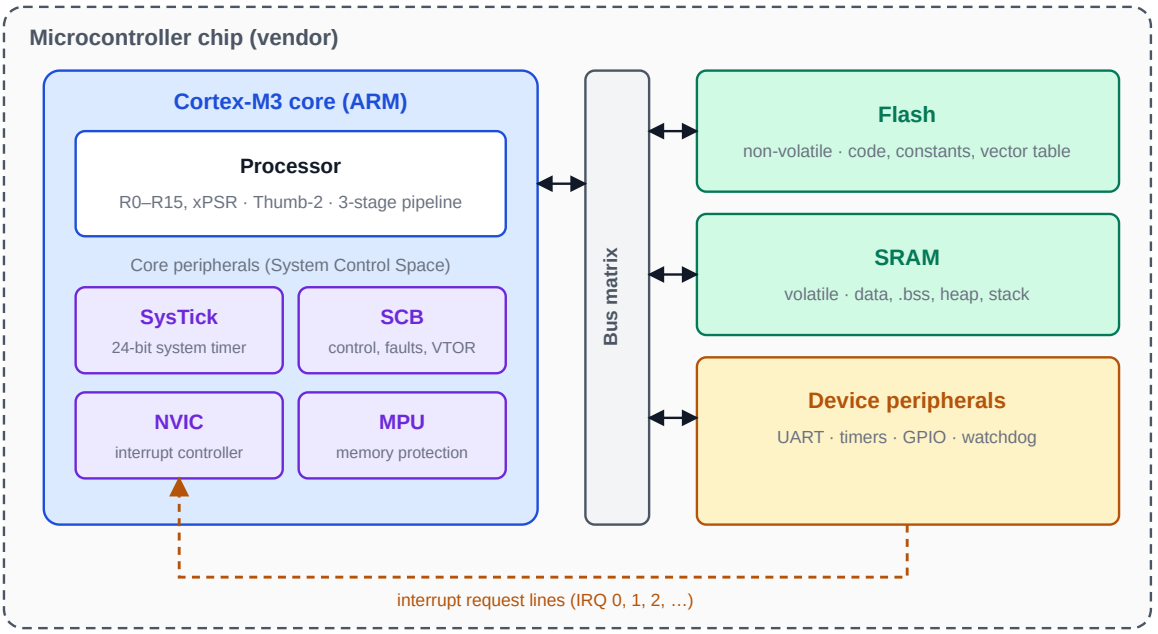


Figure 1.1: *Block diagram of a Cortex-M3 microcontroller: the ARM core with processor and core peripherals, the bus matrix, and the vendor's Flash, SRAM, and device peripherals.*

Everything in this chapter belongs to the first two layers and is therefore identical on every Cortex-M3. Board-specific values are taken from the `mps2-an385` platform (an FPGA image of a Cortex-M3 system, emulated by QEMU, clocked at 25 MHz) and are marked as such.

2 Programmer's Model and Thumb Instruction Set

The programmer's model is the set of registers visible to software and to the debugger. At this level there are no variables, only registers and memory: a C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

2.1 Core Registers

All core registers are 32 bits wide.

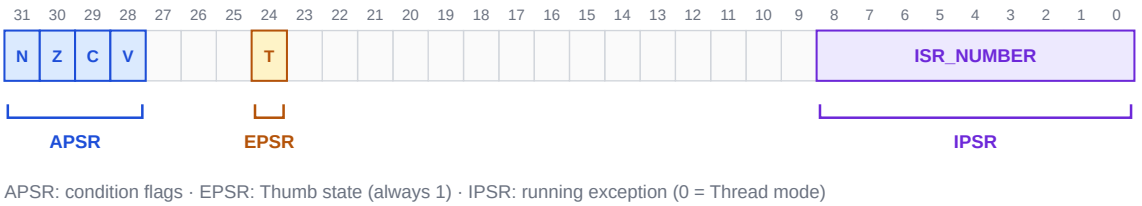
Register	Name	Role
`R0` – `R12`	General purpose	Operands and addresses; `R0` – `R3` carry function arguments
`R13`	`SP`	Stack pointer: address of the top of the stack
`R14`	`LR`	Link register: return address after a function call
`R15`	`PC`	Program counter: address of the next instruction
—	`xPSR`	Program status: flags, Thumb bit, current exception number

The stack is *full descending*: it grows towards lower addresses, and `SP` points to the last occupied word. A push first decrements `SP` by 4 and then writes at the new `SP`. `R13` is a banked name: two physical stack pointers exist (`MSP` and `PSP`), and `SP` always refers to the one currently selected (see the section on execution modes).

2.2 The Program Status Register xPSR

`xPSR` holds neither data nor addresses: it describes the current state of the processor. The name reflects that it is the union of three sub-registers occupying disjoint bits of the same 32-bit word.

Sub-register	Bits	Content
`APSR` (Application)	31–28	Flags `N` (negative), `Z` (zero), `C` (carry), `V` (overflow)
`EPSR` (Execution)	24	`T` : Thumb state, always 1 on Cortex-M
`IPSR` (Interrupt)	8–0	Number of the exception being handled, 0 if none



APSR: condition flags · EPSR: Thumb state (always 1) · IPSR: running exception (0 = Thread mode)

Figure 1.2: *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

The flags are written by comparison and arithmetic instructions and read by conditional branches; this is how every `if` and loop condition is implemented at machine level.

FLAGS IN ACTION

`cmp r1, r2` computes `r1 - r2`, discards the result, and updates only the flags: `Z = 1` if the operands are equal, `C = 1` if `r1 >= r2` as unsigned values. The following `bhs done` reads `C` and branches if it is 1.

`IPSR` identifies the execution mode: 0 means Thread mode (ordinary code), any other value is the number of the exception whose handler is running.

2.3 Load/Store Architecture

The Cortex-M3 has a load/store instruction set: arithmetic and logic operate only on registers, and memory is accessed exclusively through load (`ldr`, memory to register) and store (`str`, register to memory) instructions. Every operation on a variable therefore passes through a register.

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;` (post-increment)
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;`
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;` (any 32-bit value)
`mov sp, r0`	`sp = r0;`
`movs r3, #0`	`r3 = 0;` (small constants only)
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`b label`	`goto label;`
`bl main`	`main();` (return address saved in `LR`)
`bhs done`	`if (r1 >= r2) goto done;` after `cmp r1, r2`

Square brackets denote "the memory at this address". In `[r0], #4` the address in `r0` is used first and then incremented, so a loop walks through memory one word at a time.

An instruction is 16 or 32 bits long and cannot embed an arbitrary 32-bit constant. For the pseudo-instruction `ldr rX, =value` the assembler stores the constant in a nearby table, the *literal pool*, and emits a PC-relative load, which a disassembler shows as `ldr r0, [pc, #92]`. The `.ltorg` directive places the pool where the CPU never executes it.

```
loop:   cmp   r1, r2        @ while (r1 < r2) {
        bhs   done          @
        ...                 @     body
        b     loop          @ }
done:
```

A C `while` loop compiles to a test at the top and an unconditional branch back at the bottom: `cmp` sets the flags, `bhs` leaves the loop when the condition fails, and `b` closes the cycle.

2.4 Thumb State and the Thumb Bit

The Cortex-M executes only the Thumb-2 instruction set, a mix of 16-bit and 32-bit encodings. Because instructions are always at least 2-byte aligned, bit 0 of a code address carries no information, so ARM reuses it as the *Thumb bit*: every code pointer used in a branch-and-exchange or loaded into `PC` must have bit 0 set. The CPU checks the bit, copies it into `xPSR.T`, clears it, and jumps to the even address. A handler located at `0x40` is therefore referenced as `0x41`.

> ⚠ **THUMB BIT EQUAL TO 0**
>
> Branching to an address with bit 0 clear requests ARM state, which the Cortex-M does not implement. The result is a UsageFault (`INVSTATE`), not an ordinary crash at the target address.

2.5 Calling Convention (AAPCS)

The AAPCS (Procedure Call Standard for the Arm Architecture) defines how functions exchange arguments and which registers they must preserve. Compiled C and hand-written assembly interoperate only if both follow it, and the exception model is designed around it.

Register	Role in a call	Preserved across the call
`r0` – `r3`	First four arguments; `r0` also returns the result	No (caller-saved)
`r4` – `r11`	Long-lived local values	Yes (callee-saved)
`r12`	Scratch	No (caller-saved)
`lr`	Return address written by `bl`	No; a non-leaf function saves it
`sp`	Multiple of 8 at every public call	Yes

A caller that still needs `r0` – `r3` or `r12` after a call saves them itself; a function that uses `r4` must restore it before returning, typically with `push {r4, lr}` at entry and `pop {r4, pc}` at exit. A leaf function using only caller-saved registers saves nothing.

AAPCS: A CALL, STEP BY STEP

Calling `add3` and the code of `add3` itself:

```
int add3(int a, int b, int c);          add3:
r = add3(1, 2, 3);                          adds r0, r0, r1   /* r0 = a + b */
                                            adds r0, r0, r2   /* r0 = r0 + c */
movs r0, #1    /* a = 1 */                  bx   lr           /* return r0 to the caller */
movs r1, #2    /* b = 2 */
movs r2, #3    /* c = 3 */
bl   add3      /* lr = next instruction; pc = add3 */
               /* r = r0 */
```

- `add3` uses only r0–r2 and calls nothing, so it saves nothing: no stack at all
- A function that calls another must first save `lr` (`push {r4, lr}`), or `bl` would overwrite its own return address

`add3` is a leaf function using only caller-saved registers: it saves nothing.

3 Memory Map and Memory-Mapped Registers

The Cortex-M3 sees a single 4 GB physical address space (32-bit addresses). The architecture fixes the purpose of each region; the vendor decides how much memory or which devices populate it. There is no translation: the address issued by an instruction is the address seen by memory and peripherals, and no run-time loader chooses where a program is placed. All placement decisions are made at link time and recorded in the image.

Address range	Region	Populated on `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	4 MB program memory at `0x0` ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	4 MB at `0x2000_0000` ("RAM")
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	UARTs, timers, watchdog
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	Not used
`0xE000_0000` – `0xE00F_FFFF`	Private Peripheral Bus	Core registers (System Control Space)

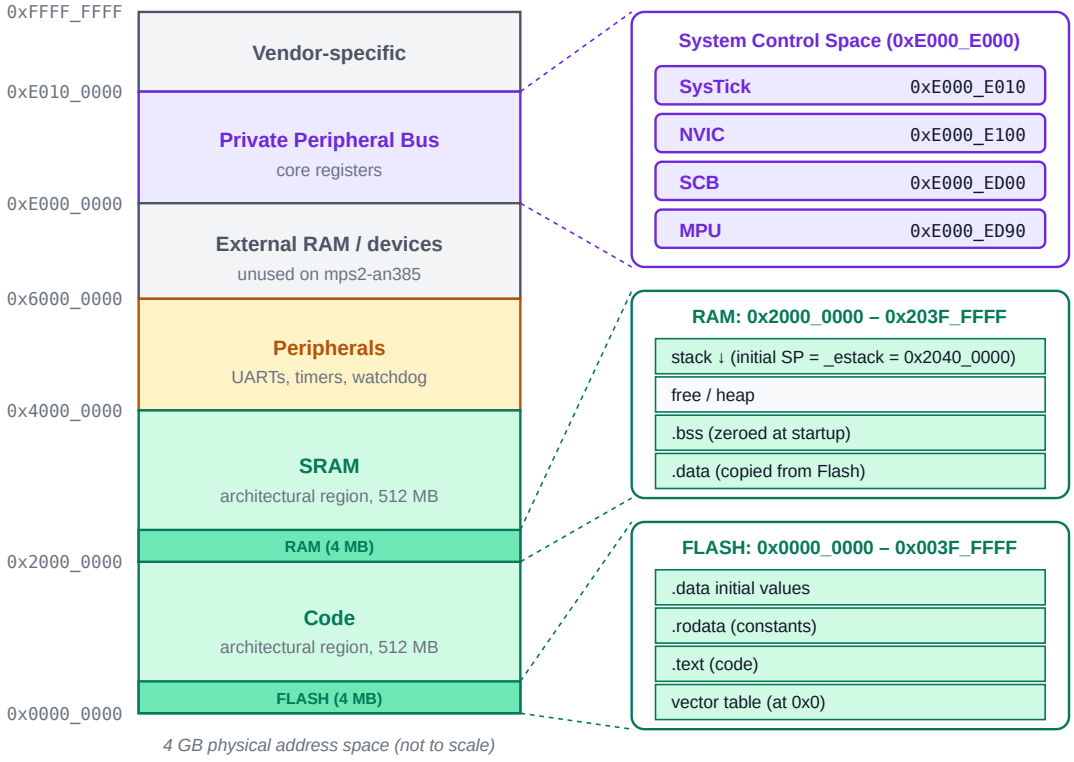


Figure 1.3: *Cortex-M3 address space with the regions populated on mps2-an385 and the System Control Space in detail.*

Flash keeps its contents without power and holds the vector table, code, constants, and the initial values of initialized variables. SRAM is fast writable memory whose contents are undefined at power-on; it holds initialized and zeroed variables, the heap, and the stack. This split is the reason startup code must copy some bytes from Flash to SRAM and zero others before C code can run safely.

> **EMULATED FLASH**
>
> On `mps2-an385`, and in QEMU, the 4 MB at address `0x0` are fast RAM preloaded with the program image. They play the role of Flash and are treated as read-only, exactly like real Flash on a microcontroller.

3.1 System Control Space and Core Peripherals

The last region holds the registers of the core itself. Inside it, the System Control Space (SCS, `0xE000_E000` – `0xE000_EFFF`) groups the core peripherals, which are identical on every Cortex-M3 regardless of the vendor.

Core peripheral	Base address	Purpose
SysTick	`0xE000_E010`	24-bit periodic timer
NVIC	`0xE000_E100`	Enabling, pending state, and priority of device interrupts
SCB (System Control Block)	`0xE000_ED00`	Vector table location, fault configuration and status, system priorities
MPU	`0xE000_ED90`	Memory protection regions

Only privileged code may access the SCS. An unprivileged access raises a BusFault, which is what prevents application code from reconfiguring the machine.

3.2 Accessing Memory-Mapped Registers

```
#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)

SCB_SHCSR |= (1u << 16);
```

```
        ldr r0, =0xE000ED24     @ address
        ldr r1, [r0]            @ read
        orr r1, r1, #0x10000    @ modify
        str r1, [r0]            @ write
```

A peripheral register is accessed with ordinary load and store instructions through a pointer to its documented address. The macro casts the address to a pointer to `volatile uint32_t` and dereferences it, so `SCB_SHCSR` behaves like a variable located at `0xE000ED24`. The compound assignment compiles to the four instructions shown: read the current value, set only the required bit, write the word back. This *read-modify-write* pattern preserves every other bit of the register and is mandatory whenever a register mixes configuration bits with status bits.

The `volatile` qualifier forces the compiler to perform every access, in program order. Without it, the compiler may assume that a value read twice cannot have changed, or that a value written and never read back is useless, and remove the access; both assumptions are false for hardware, where a read may observe a changing state and a write may trigger an external effect.

Not every register needs read-modify-write. Registers with *write-1-to-set* or *write-1-to-clear* semantics (such as the NVIC enable registers) ignore 0 bits, so a plain assignment affects only the selected bit. Registers organized

as independent bytes (such as the priority registers) are written with a byte store (`strb`).

> ⚠ **WHAT `VOLATILE` DOES NOT DO**
>
> `volatile` guarantees that accesses happen and in which order. It does not make a read-modify-write sequence indivisible: an interrupt arriving between the load and the store can still cause a lost update.

4 Execution Modes, Privilege, and Stacks

The Cortex-M3 implements in hardware the separation between kernel mode and user mode found in operating systems. Two independent properties describe the current state: the *mode* (what kind of code runs) and the *privilege level* (what that code may do).

4.1 Thread Mode and Handler Mode

Thread mode runs ordinary code: `Reset_Handler`, `main()`, and the functions they call; `IPSR` is 0. Handler mode runs exception handlers; `IPSR` holds the exception number. The CPU enters Handler mode automatically when it takes an exception and returns to the previous mode when the handler finishes.

Mode	Privilege	Typical code
Thread	Privileged	`Reset_Handler`, kernel code after reset
Thread	Unprivileged	Application tasks after the kernel drops privilege
Handler	Always privileged	Every exception handler

ONLY THREE COMBINATIONS

Privilege can be lowered only in Thread mode. Handler mode is always privileged, so an exception always runs privileged and returns to Thread mode with the privilege level it had before.

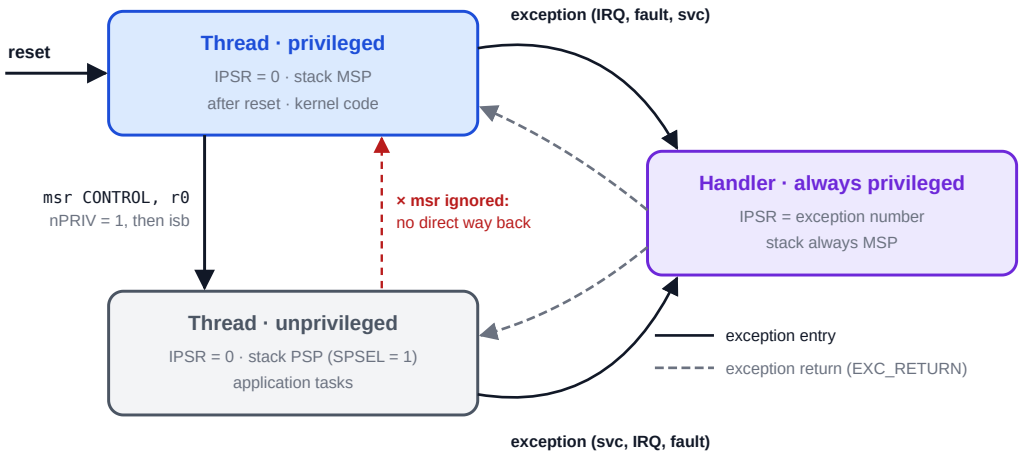


Figure 1.4: *Transitions between privileged Thread mode, unprivileged Thread mode, and Handler mode.*

After reset the CPU is in privileged Thread mode, which is why the boot code can configure the vector table, fault handling, and priorities.

4.2 Special Registers and the CONTROL Register

Some core registers are not memory-mapped and cannot be reached with `ldr` or `str`. They are read with `mrs` (move register from special) and written with `msr` (move special from register).

Special register	Content
`CONTROL`	Thread-mode privilege and stack selection
`PRIMASK`	1 bit: masks every exception with configurable priority
`FAULTMASK`	1 bit: also masks HardFault; only NMI is taken
`BASEPRI`	Masks exceptions whose priority value is greater than or equal to it

`CONTROL` holds two bits relevant here:

Bit	Name	Meaning
0	`nPRIV`	0 = privileged, 1 = unprivileged (Thread mode only)
1	`SPSEL`	Thread-mode stack pointer: 0 = `MSP`, 1 = `PSP`

4.3 Dropping Privilege

```
        mrs r0, control     @ r0 = CONTROL
        orr r0, r0, #1      @ set nPRIV
        msr control, r0     @ Thread mode is now unprivileged
        isb                 @ discard instructions fetched under the old rules
```

The sequence is a read-modify-write on a special register, equivalent to `CONTROL = CONTROL | 1`. The final `isb` (instruction synchronization barrier) is required because the CPU prefetches instructions: some may already have been fetched and decoded under the privileged rules, and `isb` flushes the pipeline so that the following instructions execute under the new ones.

Writing `CONTROL` is itself a privileged operation: an unprivileged `msr control, r0` that tries to clear `nPRIV` is silently ignored. The only ways back to privileged execution are a reset or an exception, whose handler runs in Handler mode. Unprivileged code keeps access to ordinary RAM, Flash, and its own code, but loses the ability to reconfigure or silence the machine:

Attempt by unprivileged code	Result
Access to the SCS (SysTick, NVIC, SCB, MPU)	BusFault
`msr control, r0` to regain privilege	Ignored
`cpsid i` / `cpsie i`	Ignored
Access to a region denied by the MPU	MemManage fault
Semihosting `bkpt 0xab`	Refused by QEMU by default: the fault escalates to HardFault

4.4 Main and Process Stack Pointers

The Cortex-M3 has two physical stack pointers. The **main stack pointer** (`MSP`) is used after reset and always in Handler mode; it is the kernel's stack. The **process stack pointer** (`PSP`) is used in Thread mode when `CONTROL.SPSEL` is 1; an operating system gives each task its own `PSP` stack. A task therefore typically runs with

`CONTROL = 3`: unprivileged and on `PSP`.

The register name `sp` always refers to the stack pointer currently selected, while `mrs r0, msp` and `mrs r0, psp` name each one explicitly. Because Handler mode always switches to `MSP`, interrupt handlers and system calls never execute on a task's stack: a task that corrupts or overflows its own stack cannot corrupt the stack the kernel runs on.

4.5 Interrupt Masking with PRIMASK

`cpsid i` sets `PRIMASK` to 1: every exception with configurable priority is held back, and only NMI and HardFault are taken. `cpsie i` clears it. Exceptions that arrive while masked are not lost: they remain pending and are taken as soon as `PRIMASK` returns to 0. `PRIMASK` is 0 at reset; boot code usually sets it first and clears it only once the vector table, handlers, and priorities are in place.

Instruction	`PRIMASK`	CMSIS function
`cpsid i`	1: masked	`__disable_irq()`
`cpsie i`	0: accepted	`__enable_irq()`

Critical sections that may nest must save the previous value with `mrs r0, primask` and restore it with `msr primask, r0`, rather than unconditionally executing `cpsie i`, which would unmask interrupts too early when the outer section is still active.

5 Exception Model

An *exception* is any event that makes the CPU suspend the current code and run a specific handler in Handler mode. An *interrupt* is an exception raised by a hardware device. Faults, system calls, the system timer, and device interrupts all go through the same mechanism.

5.1 Exception Types and Numbers

Number	Exception	Raised by
1	Reset	Power-on, reset pin
2	NMI	Non-maskable interrupt (e.g. watchdog expiry)
3	HardFault	Any fault with nowhere else to go
4	MemManage	Violation of MPU rules
5	BusFault	Access to an address that does not answer or refuses the access
6	UsageFault	Undefined opcode, Thumb bit 0, division by zero (if trapped)
7–10, 13	Reserved	—
11	SVCall	The `svc` instruction
12	DebugMonitor	Debug events in monitor mode
14	PendSV	A software request (`ICSR.PENDSVSET`)
15	SysTick	The core timer
16 + n	IRQ n	Device interrupt line n, through the NVIC

Numbers 0–15 belong to the core and are identical on every Cortex-M3; device interrupts follow. Number 0 is not an exception: its vector table slot holds the initial stack pointer.

At reset only HardFault is active among the faults: MemManage, BusFault, and UsageFault are disabled, so every fault *escalates* to HardFault until the boot code enables the dedicated handlers.

PendSV deserves a note because of its role in operating systems. It is raised only by software, by writing a bit in a system register, and it is normally given the lowest priority. An OS uses it to request work, typically a context switch, that must run only after every more urgent handler has finished.

5.2 Priorities, Preemption, and Lockup

When several exceptions compete, the more urgent one runs first. On Cortex-M a **lower number means more urgent**. Reset (−3), NMI (−2), and HardFault (−1) have fixed priorities above anything configurable; every other exception has a one-byte configurable priority. A pending exception preempts the running code only if its priority is *strictly* more urgent; otherwise it stays pending until the current handler returns. Preemption of one handler by another is called *nesting*, which gives the NVIC (Nested Vectored Interrupt Controller) its name.

Implementations do not use all eight bits of a priority byte. Real chips implement only the upper 3 or 4 bits, and the lower bits read as 0, so distinct priorities must differ in their top bits (`0x00` , `0x20` , ..., `0xE0` for 3 bits).

> **LOCKUP**
>
> If a fault occurs while the CPU already runs at HardFault priority, or the fault cannot be reported because exception entry itself fails (for example, the stack pointer points outside RAM), there is nowhere to escalate. The CPU enters *lockup* and stops executing. QEMU reports it as `Lockup: can't escalate 3 to HardFault (current priority -1)`.

5.3 Vector Table and Reset Sequence

The vector table is an array of 32-bit words where slot N holds the address of the handler of exception N. When exception N occurs, the CPU reads the slot and branches there. Slot 0 is special: it holds the initial value of `MSP`, not a code address. The table is located through the `VTOR` register, which resets to `0x0000_0000`; at reset it must therefore be at the start of the Code region, which is why the linker places it first in Flash.

When the Cortex-M3 leaves reset, before any software runs, the hardware performs exactly two reads:

1. the word at address `0x0` is loaded into `MSP`;
2. the word at address `0x4` is loaded into `PC`, and execution starts there in privileged Thread mode.

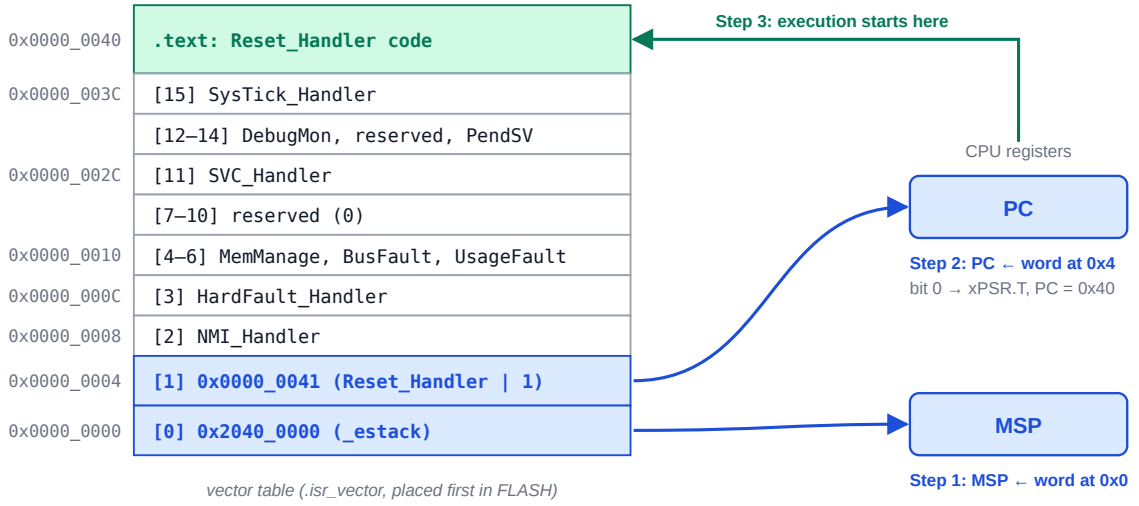


Figure 1.5: *Reset sequence: the CPU loads MSP from address 0x0 and PC from address 0x4, then executes Reset_Handler.*

There is no BIOS, no loader, and no C runtime. Initializing memory, configuring the core, and calling `main()` are the job of the code whose address is stored in slot 1, `Reset_Handler`.

Offset	Slot	Content
`0x00`	0	Initial `MSP` (e.g. `0x2040_0000`, one byte past the end of RAM)
`0x04`	1	`Reset_Handler` address, with bit 0 set
`0x08`	2	`NMI_Handler`
`0x0C` – `0x18`	3–6	HardFault, MemManage, BusFault, UsageFault handlers

Offset	Slot	Content
`0x2C`	11	`SVC_Handler`
`0x38`, `0x3C`	14, 15	`PendSV_Handler`, `SysTick_Handler`
`0x40` + 4n	16 + n	Handler of IRQ n

Every handler address is stored with the Thumb bit set. Reserved slots hold 0, and every other slot must point to valid code even if the exception is never expected; startup code usually fills unused slots with a default handler that loops forever, so that an unexpected exception stops at a recognizable address.

FROM IRQ NUMBER TO VECTOR SLOT

IRQ n is exception 16 + n, and its handler address is at offset 4 × (16 + n). IRQ 1 is exception 17, at offset `0x44`.

5.4 Exception Entry and the Stacked Frame

Before the first instruction of a handler, the hardware:

1. pushes 8 words, the *stacked frame*, onto the stack in use when the exception occurred (`MSP` or `PSP`);
2. writes a special `EXC_RETURN` value into `lr`;
3. reads the handler address from the vector table and branches to it in Handler mode, on `MSP`.

Offset from new `sp`	Stacked register
`+0x00`	`r0`
`+0x04`	`r1`
`+0x08`	`r2`
`+0x0C`	`r3`
`+0x10`	`r12`
`+0x14`	`lr` of the interrupted code
`+0x18`	`PC`: where the interrupted code resumes
`+0x1C`	`xPSR`

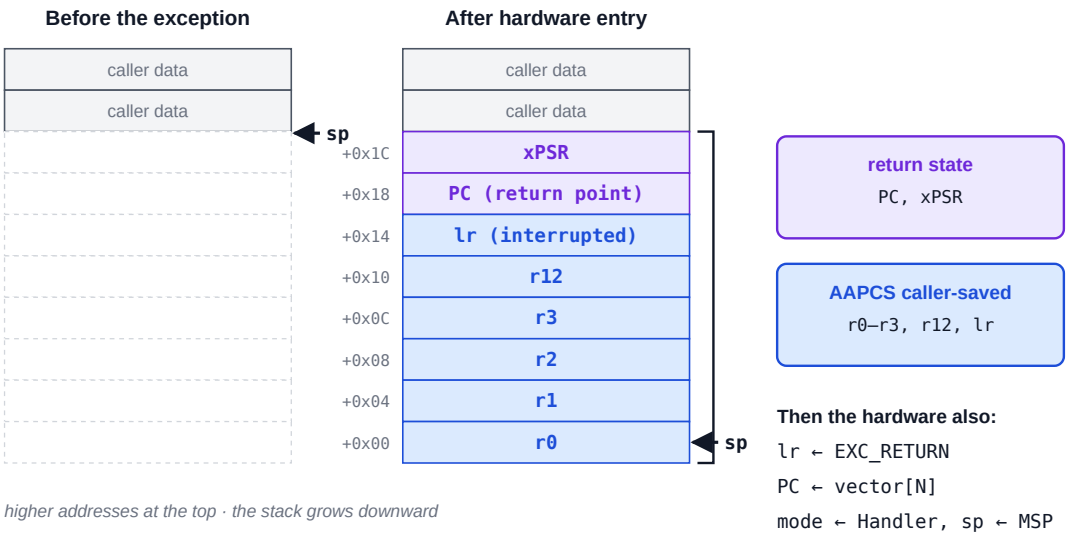


Figure 1.6: *Stack before and after exception entry, with the 8-word stacked frame pushed by hardware.*

The hardware saves exactly the AAPCS caller-saved registers plus `PC` and `xPSR`, and any compiled C function already preserves the callee-saved `r4`–`r11`. Together, these two facts mean that an exception handler can be an ordinary C function, with no special prologue or epilogue. When `CCR.STKALIGN` is set (its reset value on the Cortex-M3), the hardware also aligns `sp` to 8 bytes on entry, as the AAPCS requires, and records the adjustment in bit 9 of the stacked `xPSR`. Entry takes about 12 cycles.

5.5 Exception Return through EXC_RETURN

A handler ends with an ordinary return instruction (`bx lr` or `pop {..., pc}`), yet the CPU must restore the saved state rather than jump to a return address. While a handler runs, `lr` holds a value of the form `0xFFFFFFxx`, where no code can exist. Loading such a value into `PC` is recognized as an *exception return*: the CPU pops the 8-word frame from the indicated stack, restores the mode, and resumes at the stacked `PC`.

`EXC_RETURN`	Return to	Frame popped from
`0xFFFFFFF1`	Handler mode (a preempted handler)	`MSP`
`0xFFFFFFF9`	Thread mode	`MSP`
`0xFFFFFFFD`	Thread mode	`PSP`

Bits 31–4 are all ones and mark the value as `EXC_RETURN`. Bit 3 selects the return mode (1 = Thread, 0 = Handler) and bit 2 the stack (1 = `PSP`, 0 = `MSP`). On entry, bit 2 therefore tells the handler which stack holds the frame just pushed, which a system-call handler needs in order to read the caller's arguments.

When you return from an exception, the Linker Register is not containing the return address but also a special code 0xF

In which execution mode the CPU should go back

EXCEPTION RETURN: `EXC_RETURN`

In a handler, `lr` does not hold a code address: it holds a value starting with `0xFFFFFF..`, where there is no code. When the handler returns with `bx lr` (or `pop {..., pc}`), the CPU sees this value and does an **exception return**: it pops the 8 words, goes back to the old mode, and resumes at the stacked PC.

`EXC_RETURN`	Return to	Stack used
`0xFFFFFFF9`	thread mode (normal code, e.g. `main`)	main stack
`0xFFFFFFF1`	handler mode (a less urgent handler that was interrupted)	main stack
`0xFFFFFFFD`	thread mode	process stack

- Cortex-M has a **second stack pointer** (the process stack). This kernel never uses it, so a return to thread mode here is always `0xFFFFFFF9`
- The compiled C handler ends with an ordinary `bx lr` or `pop {..., pc}`: the value in `lr` is what makes it an exception return

5.6 Tail-Chaining and Late Arrival

When exceptions occur close together, the core avoids saving and restoring the same state repeatedly.

Optimization	Situation	Behaviour
Tail-chaining	A handler ends while another exception is pending	No pop and re-push of the frame: direct jump to the next handler (about 6 cycles instead of about 24)
Late arrival	A more urgent exception arrives while the frame of a less urgent one is being pushed	The frame is reused; the more urgent handler runs first, then tail-chains to the other

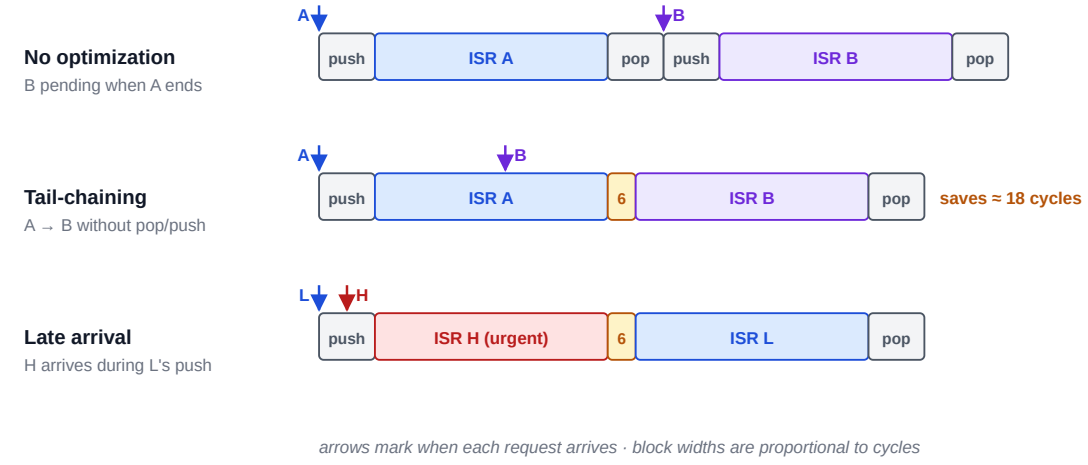


Figure 1.7: *Timelines of two exceptions without optimization, with tail-chaining, and with late arrival.*

5.7 SVC: A Controlled Entry into Privileged Code

```c
void SVC_Handler( void )
{
    /* privileged service code */
}

/* in unprivileged Thread mode */
__asm__ volatile ( "svc #0" : : : "memory" );
```

`svc #n` (supervisor call) is one of the few things unprivileged code can always do. It raises SVCall (exception 11) synchronously, so the handler in slot 11 runs in Handler mode and therefore privileged. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and still unprivileged. Inside the handler `CONTROL.nPRIV` still reads 1: the handler is privileged because of the mode, not because the bit changed.

Privilege is borrowed only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction and, together with arguments in `r0` – `r3`, can select the requested service. This is the hardware foundation of every system call: restricted code cannot touch the machine directly and must go through a single, controlled entry point.

> ⚠ **MISSING HANDLER**
>
> If `SVC_Handler` is not defined, `svc` reaches the default handler and the program hangs silently. Unlike a fault handler, `SVC_Handler` returns normally, because `svc` is a deliberate request, not an error.

6 Configuring the Core: SCB, NVIC, and SysTick

The behaviour of the exception model is configured through memory-mapped registers in the System Control Space. All of them are accessible only from privileged code.

6.1 System Control Block Registers

Register	Address	Role
`CPUID`	`0xE000_ED00`	Core identification (`0x410FC231` : ARM, Cortex-M3)
`ICSR`	`0xE000_ED04`	Pending state of system exceptions; `PENDSVSET` (bit 28) raises PendSV
`VTOR`	`0xE000_ED08`	Address of the vector table
`CCR`	`0xE000_ED14`	Which conditions are treated as faults
`SHPR1` – `SHPR3`	`0xE000_ED18` – `0xE000_ED20`	Priorities of system exceptions
`SHCSR`	`0xE000_ED24`	Enable and status of system handlers
`CFSR`	`0xE000_ED28`	Cause of MemManage, BusFault, UsageFault
`HFSR`	`0xE000_ED2C`	Cause of HardFault
`MMFAR`	`0xE000_ED34`	Address that caused a MemManage fault
`BFAR`	`0xE000_ED38`	Address that caused a BusFault

`VTOR`. On an exception the CPU reads the handler address from `VTOR + 4 × N`. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes (more when many IRQs exist). Writing `VTOR` is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a bootloader at address 0 can start an application stored higher in Flash, which then points `VTOR` at its own table, or a program can copy its table into RAM to replace handlers at run time.

`SHCSR`. Bits 16 (`MEMFAULTENA`), 17 (`BUSFAULTENA`), and 18 (`USGFAULTENA`) enable the dedicated fault handlers. At reset they are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. The other bits report whether handlers are active or pending, so the register is updated with read-modify-write (`|= 0x00070000`), never overwritten.

`CCR`. The configuration bits relevant here are listed below.

Bit	Name	Effect when set
3	`UNALIGN_TRP`	Unaligned word and halfword accesses raise a UsageFault
4	`DIV_0_TRP`	Integer division by zero raises a UsageFault
9	`STKALIGN`	8-byte stack alignment on exception entry (1 at reset)

Without `DIV_0_TRP` , `sdiv` and `udiv` silently return 0 when the divisor is 0 and the program continues with a wrong value; setting the bit turns this silent bug into a fault. Unaligned accesses are legal on the Cortex-M3, only

slower, and trapping them can break correct compiler-generated code, so `UNALIGN_TRP` is usually left off. Since `STKALIGN` is already 1, enabling the division trap requires read-modify-write, giving `0x210`.

`SHPR1` – `SHPR3`. Each system exception has one priority byte, so the three registers can be viewed as a 12-byte array starting at `0xE000_ED18`, indexed by exception number minus 4. Each byte is written with a byte store and needs no read-modify-write.

Register	Byte 0	Byte 1	Byte 2	Byte 3
`SHPR1`	MemManage	BusFault	UsageFault	—
`SHPR2`	—	—	—	SVCall
`SHPR3`	—	—	PendSV	SysTick

A typical policy makes faults most urgent (`0x00`), system calls intermediate (`0x80`), and PendSV and SysTick least urgent (`0xE0`). Reading the words as little-endian, this gives `SHPR2 = 0x80000000` and `SHPR3 = 0xE0E00000`.

6.2 Fault Status Registers

```
void UsageFault_Handler( void )
{
    kprintf( "UsageFault: CFSR=0x%x\n", SCB_CFSR );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    for ( ;; ) { }
}
```

The divisor is `volatile`, so the compiler cannot evaluate the division at build time and must emit a real `udiv` instruction. With `CCR.DIV_0_TRP` and `SHCSR.USGFAULTENA` set, the instruction raises a UsageFault, the dedicated handler runs, and it prints `CFSR=0x2000000`: bit 25, `DIVBYZERO`. The handler then loops forever, because a fault is a bug with no sensible way to continue. Removing each configuration step isolates its role:

Configuration	Observed behaviour
`SHCSR` bits and `DIV_0_TRP` set	`UsageFault_Handler` runs and reports the cause
`SHCSR` bits not set	The fault escalates to HardFault; its kind is lost
`DIV_0_TRP` not set	No fault: the division returns 0 and `main` continues

`CFSR` is the concatenation of three status registers, one per configurable fault. Its most useful bits are:

Bits	Sub-register	Main flags
7–0	MMFSR (MemManage)	`IACCVIOL` (0), `DACCVIOL` (1), `MUNSTKERR` (3), `MSTKERR` (4), `MMARVALID` (7)
15–8	BFSR (BusFault)	`PRECISERR` (9), `IMPRECISERR` (10), `BFARVALID` (15)
31–16	UFSR (UsageFault)	`UNDEFINSTR` (16), `INVSTATE` (17), `UNALIGNED` (24), `DIVBYZERO` (25)

`HFSR` bit 30 (`FORCED`) indicates that the HardFault is an escalated configurable fault; the original cause is then found in `CFSR` . The bits are sticky and are cleared by writing 1 to them.

6.3 NVIC: Device Interrupts

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

Device interrupt requests do not reach the CPU directly: they pass through the NVIC, which numbers them, enables them, and assigns their priority. The first line writes the priority byte of IRQ 1 with a byte store. The second sets bit `n % 32` of word `n / 32` in the enable register; since set and clear registers ignore 0 bits, a plain assignment enables one line without disturbing the others.

Register	Base address	Bit or byte for IRQ n	Write semantics
`ISER`	`0xE000_E100`	Bit `n % 32` of word `n / 32`	1 = enable
`ICER`	`0xE000_E180`	Same	1 = disable
`ISPR`	`0xE000_E200`	Same	1 = set pending
`ICPR`	`0xE000_E280`	Same	1 = clear pending
`IPR`	`0xE000_E400`	Byte n	Priority; lower = more urgent

A device interrupt runs its handler only if all of the following hold:

1. the device is configured to raise it (an interrupt-enable bit in the device's own control register);
2. the NVIC line is enabled in `ISER` ;
3. its priority is more urgent than that of the running code, and `PRIMASK` is 0.

NVIC lines driven by device flags are level-sensitive: as long as the device keeps its interrupt flag set, the request stays active. A handler must therefore *acknowledge* the interrupt at the device, typically by writing 1 to clear the flag, otherwise the handler is re-entered forever as soon as it returns.

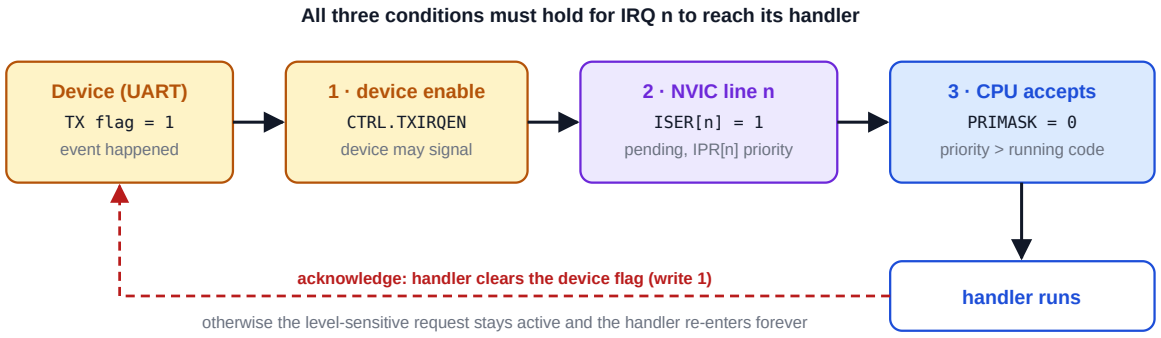


Figure 1.8: *Path of a device interrupt through the device enable, the NVIC, and the CPU masking and priority check, with acknowledgement at the device.*

6.4 SysTick: The Core Timer

SysTick is a 24-bit down-counter present in every Cortex-M core. It counts from the reload value to 0, reloads, and can raise exception 15 each time it reaches 0. An operating system uses this periodic exception, the *tick*, as its time base.

Register	Address	Content
`CTRL`	`0xE000_E010`	`ENABLE` (0), `TICKINT` (1), `CLKSOURCE` (2), `COUNTFLAG` (16, cleared on read)
`LOAD`	`0xE000_E014`	24-bit reload value (max `0xFFFFFF`)
`VAL`	`0xE000_E018`	Current count; any write clears it
`CALIB`	`0xE000_E01C`	Read-only; `TENMS` = reload value for 10 ms on the reference clock

`CLKSOURCE` selects between the processor clock (1) and an implementation-defined reference clock (0). On `mps2-an385` the reference clock runs at 1 MHz (`CALIB.TENMS = 9999`) and the processor clock at 25 MHz. `CALIB` serves portable code that does not know the processor frequency; when the frequency is known, the reload value is computed directly. Writing `CTRL = 7` starts the counter on the processor clock with the exception enabled.

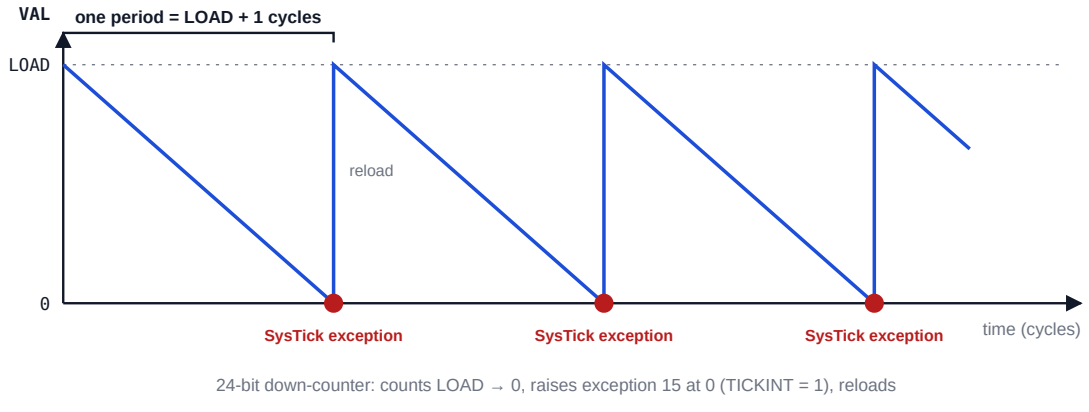


Figure 1.9: *SysTick counter value over time: a sawtooth from LOAD to 0 with an exception at every zero.*

Since one period lasts `LOAD + 1` cycles, the reload value for a tick rate r at clock frequency f is

$$\mathrm{LOAD} = \frac{f}{r} - 1$$

At 25 MHz and 100 ticks per second, `LOAD = 249 999` (`0x3D08F`), which fits in 24 bits; the longest possible period is 2^{24} cycles, about 0.67 s. A faster tick gives finer time resolution at the cost of more exceptions per second.

SysTick is a core exception, not a device interrupt, and this changes how it is configured and handled:

Aspect	SysTick	Device interrupt
Priority set in	`SHPR3`	NVIC `IPR`
Enabled by	`CTRL.TICKINT`	Device enable bit and NVIC `ISER`
Acknowledge in the handler	Not needed: pending state clears on entry	Required, otherwise the handler is re-entered

7 Memory Protection Unit

The Cortex-M3 has no MMU. What it offers instead is an optional **Memory Protection Unit** (MPU), present on `mps2-an385` and disabled after reset.

7.1 MPU Versus MMU

An MMU *translates* every virtual address into a physical one through page tables, and checks permissions as part of the translation. An MPU *only checks*: on every load, store, and instruction fetch, it compares the address with a short list of regions. If the access is allowed, it proceeds at the same physical address; if not, it does not happen at all (a store writes nothing, a load reads nothing) and the CPU raises a MemManage fault. Every task sees real physical addresses, so the linker must place each task at its own address.

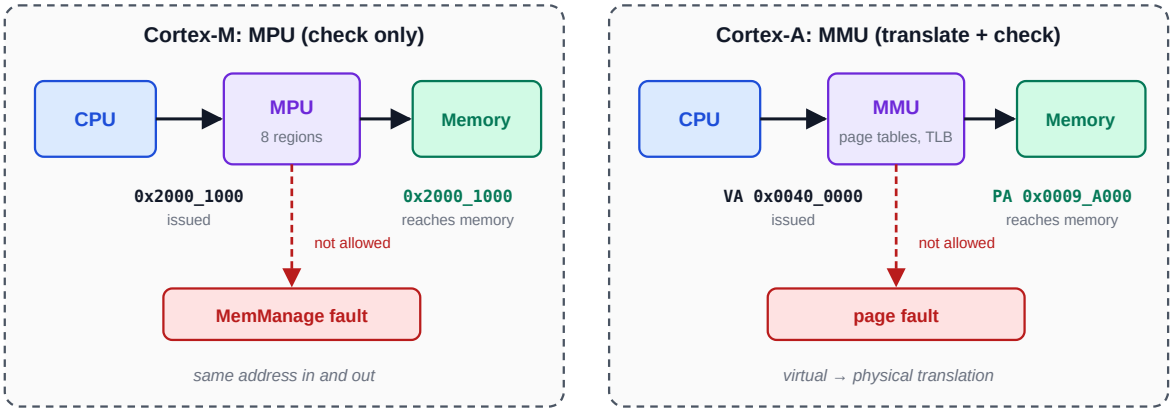


Figure 1.10: *Comparison of an MPU, which only checks addresses, and an MMU, which translates virtual to physical addresses.*

Microcontrollers adopt the MPU for reasons that follow directly from embedded constraints. An MPU is a handful of region registers, while an MMU needs page tables in RAM, a hardware table walker, and a TLB, which cost silicon area and energy. An MPU check takes the same time on every access, whereas a TLB miss makes some accesses slower by an amount that depends on the TLB contents, which harms timing predictability. Finally, with no disk and a small fixed set of tasks, virtual memory would bring no benefit.

7.2 Regions and Permissions

The Cortex-M3 MPU has 8 regions, numbered 0 to 7 (the count is read from `MPU_TYPE` bits 15–8). Each region has a base address, a size, permissions, and an enable bit, subject to two rules:

- the size is a power of two, from 32 bytes to 4 GB;
- the base address is a multiple of the size, i.e. the region is aligned to its own size.

A 4 KB region can start at `0x2000_1000` but not at `0x2000_0400`. A task that needs 5 KB receives an 8 KB region, the next power of two.

The **AP** (access permission) field encodes two rights at once, one for privileged and one for unprivileged code:

AP	Privileged	Unprivileged
`000`	No access	No access
`001`	Read/write	No access
`010`	Read/write	Read-only
`011`	Read/write	Read/write
`110`	Read-only	Read-only

The **XN** (execute never) bit forbids instruction fetches from a region while still allowing reads and writes if AP permits them. Marking data and stack regions XN prevents code injection: a task cannot execute machine code it has written into a buffer.

The **PRIVDEFENA** bit of `MPU_CTRL` decides what happens to privileged accesses that hit no region. When it is 1, they use the default memory map as if the MPU were off, so the kernel keeps access to all memory and peripherals and only unprivileged code is restricted. When it is 0, even privileged code may access only what the regions allow. An unprivileged access that hits no enabled region is always refused.

7.3 MPU Registers and Region Encoding

Register	Address	Fields
`MPU_TYPE`	`0xE000_ED90`	Bits 15–8: number of regions
`MPU_CTRL`	`0xE000_ED94`	`ENABLE` (0), `PRIVDEFENA` (2)
`MPU_RNR`	`0xE000_ED98`	Region selected by `RBAR` and `RASR`
`MPU_RBAR`	`0xE000_ED9C`	Base address (31–5), `VALID` (4), `REGION` (3–0)
`MPU_RASR`	`0xE000_EDA0`	`XN` (28), `AP` (26–24), `S` / `C` / `B` (18–16), `SIZE` (5–1), `ENABLE` (0)

A single `RBAR` / `RASR` pair serves all eight regions; `RNR` selects which one it refers to. When `RBAR` is written with its `VALID` bit set, the region number is taken from its own `REGION` field, so one write replaces the pair of writes to `RNR` and `RBAR`. The low 5 bits of a base address are always 0, because the base is aligned to at least 32 bytes, which leaves room for `VALID` and `REGION` in the same word.

The `SIZE` field holds $\log_2(\text{size}) - 1$, so a region spans $2^{\text{SIZE}+1}$ bytes; the offset of 1 lets 5 bits reach 4 GB with `SIZE = 31`. The `S`, `C`, and `B` bits describe the memory type for caches and multi-core systems and have no functional effect on a core without cache.

```
#define MPU_CTRL (*(volatile uint32_t *)0xE000ED94)
#define MPU_RBAR (*(volatile uint32_t *)0xE000ED9C)
#define MPU_RASR (*(volatile uint32_t *)0xE000EDA0)

void mpu_setup( void )
{
    MPU_RBAR = 0x00000000u | 0x10u | 0u;  /* region 0: Flash, VALID */
    MPU_RASR = 0x0602002Bu;               /* AP=110, C, 4 MB, enabled */
```

```
    MPU_RBAR = 0x20001000u | 0x10u | 1u;  /* region 1: task partition, VALID */
    MPU_RASR = 0x13060017u;               /* XN, AP=011, S, C, 4 KB, enabled */
    MPU_CTRL = ( 1u << 2 ) | 1u;          /* PRIVDEFENA | ENABLE */
    __asm volatile ( "dsb\n\tisb" : : : "memory" );
}
```

The function configures a typical two-region layout for running one unprivileged task. Region 0 covers the 4 MB of Flash as read-only and executable for everyone, so the task can run its code and read constants. Region 1 covers the 4 KB partition of the running task as read/write and non-executable. Everything else (other tasks' memory, kernel data, kernel stack, peripherals) has no region and is therefore forbidden to unprivileged code, while `PRIVDEFENA` leaves the kernel unrestricted. A kernel that switches tasks only needs to rewrite region 1. The final `dsb` waits for all register writes to complete and `isb` ensures that the following instructions are checked against the new regions.

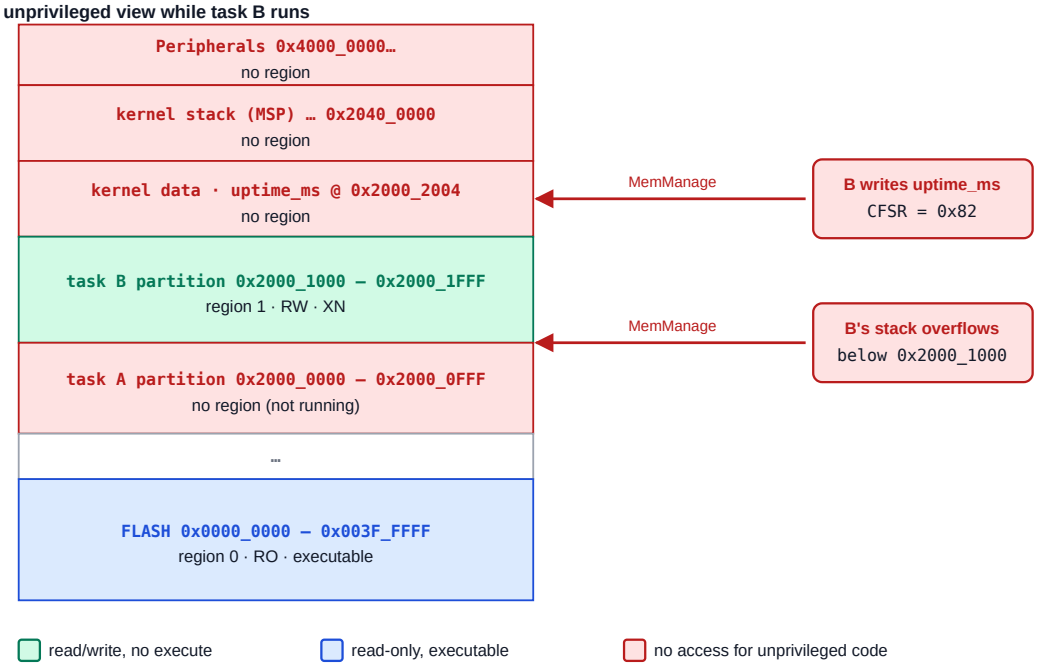
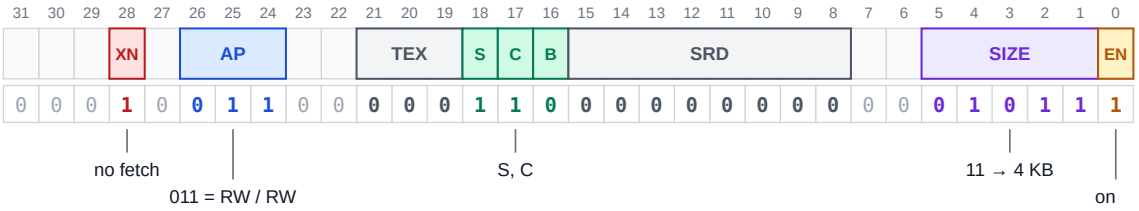


Figure 1.11: *Unprivileged view of memory with the MPU enabled while task B runs: Flash region 0, task B partition region 1, everything else forbidden.*

The two `RASR` values decompose as follows:

Field	Region 0 (Flash)	Region 1 (partition)
`XN` (bit 28)	0	1 → `0x10000000`
`AP` (bits 26–24)	`110` → `0x06000000`	`011` → `0x03000000`
`S`, `C` (bits 18, 17)	`C` → `0x00020000`	`S`, `C` → `0x00060000`
`SIZE` (bits 5–1)	21 (4 MB = 2^{22}) → `0x2A`	11 (4 KB = 2^{12}) → `0x16`
`ENABLE` (bit 0)	`0x01`	`0x01`
Total	`0x0602002B`	`0x13060017`



RASR = 0x1306_0017: region 1 (task partition) · TEX and SRD left at 0

Figure 1.12: *Field layout of MPU_RASR with the value 0x13060017 decoded bit by bit.*

7.4 MemManage Faults

A refused access raises MemManage (exception 4), which reaches its own handler only if `SHCSR.MEMFAULTENA` is set; otherwise it escalates to HardFault. The handler runs privileged and on `MSP`, and it finds the cause in the low byte of `CFSR`:

Bit	Name	Set when
0	`IACCVIOL`	An instruction fetch was refused (e.g. from an XN region)
1	`DACCVIOL`	A data load or store was refused
3	`MUNSTKERR`	Popping the frame on exception return was refused
4	`MSTKERR`	Pushing the frame on exception entry was refused
7	`MMARVALID`	`MMFAR` holds the refused address

Several bits can be set at once. A value of `0x82`, for example, means `MMARVALID` plus `DACCVIOL`: a data access was refused and its address is in `MMFAR`. Looking that address up in the symbol table of the image (with `nm`) names the variable the faulty code tried to touch. A task that overflows its stack below the start of its partition produces the same kind of fault, which makes the MPU an effective stack-overflow detector.

> **READ `MMFAR` ONLY WHEN `MMARVALID` IS SET**
>
> For instruction-fetch and stacking faults the CPU does not update `MMFAR`, which may then hold a stale, meaningless value.